Entrenar un LLM pequeño para resolver Sudoku: comparación de SFT, DPO y RLVR sobre un entorno verificable

Propuesta inicial – Entrega 1

Lilly Guevara Cristian Hinostroza Juan Moya Dan Villablanca

IIC-3694 Inteligencia Artificial Generativa – 11 de noviembre de 2026

Resumen

Proponemos comparar, sobre un mismo entorno de Sudoku con verificador, cuatro estrategias de post-entrenamiento de Qwen3.5-2B (solo SFT, SFT seguido de RLVR con GRPO, solo RLVR y DPO) junto con el modelo base sin entrenamiento como línea de control, con un presupuesto de cómputo comparable. Como contribución propia, proponemos un currículo de dificultad basado en el número de celdas vacías para el entrenamiento con RLVR, y lo comparamos contra entrenar con dificultades mezcladas. El entorno, los datos y la línea base preliminar provienen del Desafío. Buscamos responder si el SFT previo acelera o limita el aprendizaje con RLVR, cuánto generaliza cada estrategia a la partición de prueba y a niveles de dificultad no vistos, si un currículo por celdas vacías mejora la eficiencia y el resultado final, si lo aprendido se transfiere a tareas de otros entornos del curso, y si aparece *reward hacking*. Documentaremos el ciclo de iteración (entrenar, inspeccionar trazas, corregir el verificador y reentrenar) e incluiremos una auditoría manual de trazas exitosas.

1 Introducción y motivación

El aprendizaje reforzado con recompensas verificables (RLVR) ha pasado a ser una pieza central del post-entrenamiento de modelos de razonamiento [4, 7]. Su ingrediente clave es un verificador automático que sirve como recompensa. Esto lo hace atractivo, pero también frágil: un verificador imperfecto puede ser explotado por la política [13]. Además, hay evidencia mixta sobre si RLVR agrega capacidades o solo afina las que el modelo ya posee [18], y sobre el rol del SFT previo, que puede acelerar el aprendizaje pero también reducir la exploración.

Estas preguntas son difíciles de estudiar a escala de laboratorio de frontera, pero se pueden abordar con un modelo pequeño y un entorno barato de verificar. En el Desafío construimos un harness de Sudoku (4×4, 6×6 y 9×9, tres niveles de dificultad) cuya respuesta se verifica por construcción. El Sudoku es un problema de razonamiento con restricciones, de varios pasos, que Qwen3.5-2B resuelve mal. Este proyecto usa ese entorno para comparar estrategias de entrenamiento de forma controlada. Entrenaremos solo con 4×4 y 6×6; los 9×9 quedan fuera del entrenamiento y se usarían únicamente para evaluar.

2 Revisión bibliográfica

Entornos verificables. Reasoning Gym [14] ofrece generadores de tareas con verificadores y dificultad configurable; nuestro generador de Sudoku adapta el suyo. AgentGym [16] sigue la misma filosofía para agentes con herramientas.

Algoritmos. PPO [11] es la base de muchos métodos de RLHF. GRPO [12] elimina el crítico y estima la ventaja comparando las recompensas de un grupo de respuestas a un mismo prompt; así se usó en DeepSeek-R1 [4]. Trabajos posteriores analizan sesgos de GRPO, como la normalización por longitud y por desviación estándar [8], y proponen mejoras prácticas, como filtrar grupos sin señal y recortar de forma asimétrica [17]. DPO [10] optimiza directamente sobre pares de preferencia, sin modelo de recompensa ni muestreo en línea, y es de bajo costo. Para SFT y DPO con pocos recursos usaremos LoRA [5] sobre las implementaciones de TRL [15].

Currículo. El aprendizaje por currículo presenta ejemplos de menor a mayor dificultad [2]. En RLVR para LLM, Kimi k1.5 usa muestreo por currículo y priorizado [6], y DAPO filtra grupos donde todas las respuestas reciben la misma recompensa [17]. Nuestro entorno permite controlar la dificultad con una variable explícita y barata, el número de celdas vacías, lo que facilita estudiar el currículo de forma controlada.

Qué aporta RLVR. Yue et al. [18] reportan que, medido con `pass@`k para k grande [3], los modelos entrenados con RLVR no superan al modelo base. Esto motiva reportar `pass@`k además de `pass@1`, y estudiar en qué zona de dificultad hay señal.

Reward hacking y auditoría. Skalse et al. [13] formalizan el *reward gaming*. En código, SWE-Bench+ [1] mostró que una parte importante de los aciertos aparentes no era genuina, lo que motiva la auditoría manual de trazas exitosas que pide el enunciado [9].

3 Entorno a utilizar

Usaremos el harness del Desafío (repositorio `harness-iagen-sudoku`). El modelo objetivo es `Qwen/Qwen3.5-2B`.

- **Interfaz:** `reward(prompt, completion) -> float`. El parser exige un único bloque final `<answer>` con una matriz JSON y rechaza texto posterior.
- **Tareas:** particiones train/eval/test, generadas con solución única, tres tamaños de tablero y tres niveles por tamaño. Las particiones no comparten puzzles.
- **Recompensa:** reward = format $\cdot$ $(w_{fill} \cdot fill \cdot clues_frac + w_{solve} \cdot solved)$, con crédito parcial y reward = 1 solo si la solución es válida. Se mantiene además la variante binaria.
- **Datos derivados:** trazas correctas para SFT y pares correcta/incorrecta para DPO, generados con Qwen3.5-9B como profesor sobre la partición de entrenamiento y filtrados con el verificador. Estamos regenerándolos con la misma configuración.

Línea base preliminar. En la evaluación del Desafío ($n = 8$ muestras por tarea, 10 tareas de validación por tamaño y nivel), solo 12 de 90 tareas tuvieron resultados mixtos (algunos aciertos y algunos fallos); todas están en 4×4 y 6×6 *easy*/*medium*. En 6×6 *hard* y en todo 9×9 el modelo no acertó ninguna tarea. Esta zona sin señal, junto con el costo de generar y entrenar con 9×9, motiva descartar ese tamaño del entrenamiento (sección 4). Estamos recalculando las líneas base con una nueva versión en un cluster, por lo que estos números son provisionales y se actualizarán en la Entrega 2.

4 Propuesta de investigación

4.1 Preguntas e hipótesis

P1. ¿El SFT previo acelera o limita el aprendizaje con RLVR? *H1:* SFT→RLVR converge más rápido que solo RLVR en la zona aprendible, pero con `pass@`k final similar o menor para k grande.
P2. ¿Cuánto generaliza cada estrategia a la partición de prueba, a niveles no vistos y a un tamaño de tablero no visto (9×9)? *H2:* RLVR generaliza mejor que SFT y DPO a niveles no vistos; SFT tiende a memorizar el formato de las trazas del profesor.
P3. ¿Aparece *reward hacking* y bajo qué condiciones? *H3:* aparecerá más con recompensa parcial que con la binaria, por ejemplo respuestas que copian pistas y adivinan el resto, o que explotan el formato.
P4. ¿Un currículo por número de celdas vacías mejora el entrenamiento con RLVR frente a mezclar dificultades? *H4:* un currículo adaptativo alcanza la misma recompensa con menos pasos y llega a niveles que el entrenamiento con mezcla uniforme no resuelve (por ejemplo 6×6 *hard*), porque mantiene los grupos de GRPO en la zona con señal.
P5. ¿Lo aprendido en Sudoku se transfiere, o se pierde, en tareas de razonamiento de otros entornos? *H5:* el SFT degrada más el desempeño fuera de dominio que RLVR, que se mantiene más cerca del modelo base.

4.2 Experimento central

Los cinco caminos (el modelo base como control y cuatro estrategias de entrenamiento) se evalúan sobre el mismo entorno, con presupuesto de cómputo comparable (criterio en la sección 5).

#	Camino	Descripción
1	Base (control)	Qwen3.5-2B sin entrenamiento adicional.
2	SFT	SFT con LoRA sobre trazas correctas del profesor.
3	SFT→RLVR	Camino 2 seguido de GRPO con el verificador como recompensa.
4	RLVR	GRPO directamente desde el modelo base.
5	DPO	DPO con LoRA sobre pares correcta/incorrecta.

4.3 Diseño y ablations

- **Datos de entrenamiento y evaluación.** Se entrena solo con 4×4 y 6×6 (todos los niveles, ya que el currículo busca llegar a 6×6 *hard*); los caminos 2 a 5 usan una mezcla fija de dificultades. Los 9×9 se excluyen del entrenamiento. Por la línea base, la zona con señal es 4×4 y 6×6 *easy*/*medium*. *[Por decidir: confirmar la mezcla tras la nueva línea base]*
- **Evaluación en 9×9.** Si se mantiene, mide generalización a un tamaño no visto. Con la línea base actual se espera `pass@1` cercano a cero, por lo que podría aportar poca información. *[Por decidir: mantener 9×9 en la evaluación o descartarlo del todo]*
- **Ablations (RLVR).** Tamaño de grupo G, coeficiente KL, forma de la recompensa (binaria vs. parcial) y mezcla de dificultades. *[Por decidir: cuáles son viables con el cómputo]*
- **Contribución (nivel 3): currículo por celdas vacías.** En lugar de los tres niveles gruesos, se usa el número de celdas vacías como eje continuo de dificultad. Se entrena GRPO con tres variantes de muestreo de tareas: (a) *mezcla uniforme* sobre todo el rango (baseline); (b) *currículo fijo*, con un calendario creciente de celdas vacías; (c) *currículo adaptativo*, que avanza

al siguiente rango cuando el `pass@1` de entrenamiento reciente supera un umbral τ y retrocede si cae bajo uno inferior. Así se llega más allá de la zona actual, hasta 6×6 *hard*. La comparación es contra GRPO base, con igual cómputo, y se reportan curvas de recompensa y `pass@`k por número de celdas vacías. *[Por decidir: umbral τ, calendario, tamaño de ventana]*

- **Eje de dificultad entre tamaños.** Las celdas vacías no son comparables entre tableros (12 vacías es el máximo en 4×4, pero en 6×6 corresponde a *easy*). Opciones: currículo por tamaño y dentro de cada tamaño por celdas vacías, o un orden global por dificultad empírica (`pass@1` de la línea base). *[Por decidir: elegir el eje]*
- **Soporte en el entorno.** El generador ya produce tareas por rangos de celdas vacías (`LEVELS`) y la partición de prueba admite conteos fijos (`TEST_CELLS`). El conjunto actual (`data/sudoku`) tiene solo 50 tareas de entrenamiento por tamaño y nivel (150 por tamaño). Para un currículo fino (4×4 tiene 9 conteos de celdas y 6×6 tiene 17) hay que generar más tareas, con un nivel por conteo, y escribir un muestreador de tareas para el currículo. La partición de prueba usa conteos fijos y no cambia.
- **Métricas.** `pass@1` y `pass@`k por nivel de dificultad y por número de celdas vacías, con el estimador insesgado de Chen et al. [3], en la partición de prueba, y varias semillas (≥ 3 si el cómputo lo permite). Todos los resultados se reportan con la *versión final* del verificador.
- **Evaluación en otros entornos.** Los modelos de los cinco caminos se evaluarán, sin entrenar en ellos, con harness de otros grupos del curso, para medir transferencia u olvido respecto al modelo base. Como esos entornos se publicarán durante esta semana, los elegiremos una vez disponibles, con este criterio: (i) que respeten la interfaz común `reward(prompt, completion) -> float`; (ii) que sean de razonamiento o de uso de herramientas y no de memorización; (iii) que el modelo base tenga `pass@1` distinto de cero, para que la comparación sea informativa; y (iv) que la inferencia sea de bajo costo. Se reporta `pass@1` y `pass@`k como en Sudoku, y se compara cada camino contra el modelo base. Si no hay harness de otros grupos utilizables, usaremos como respaldo tareas públicas de Reasoning Gym [14] distintas del Sudoku. *[Por decidir: cuál o cuáles harness; cuántas tareas por entorno]*
- **Auditoría manual.** Muestra de trazas exitosas de cada camino, revisada a mano para estimar qué fracción de los aciertos es genuina (razonamiento válido), al estilo de Aleithan et al. [1]. *[Por decidir: tamaño de la muestra y quién audita]*

4.4 Ciclo de iteración y bloques del sistema

Registraremos cada iteración: qué se observó en las trazas, qué se corrigió en el entorno o el verificador y su efecto. La figura 1 resume el sistema.

5 Cómputo y riesgos

Usaremos un cluster con una GPU NVIDIA RTX A6000 (48 GB). SFT y DPO con LoRA son de bajo costo; GRPO es lo más caro, por lo que limitaremos el largo de las respuestas y el tamaño del entorno. Podemos solicitar acceso a Peteroa si se justifica.

Comparación a cómputo equivalente. Los métodos tienen estructuras de costo distintas: SFT y DPO procesan datos fijos, mientras que GRPO genera G respuestas por prompt y luego actualiza. Por ello proponemos:

- *Presupuesto común B* en tokens procesados por el modelo objetivo (generados y de entrenamiento)

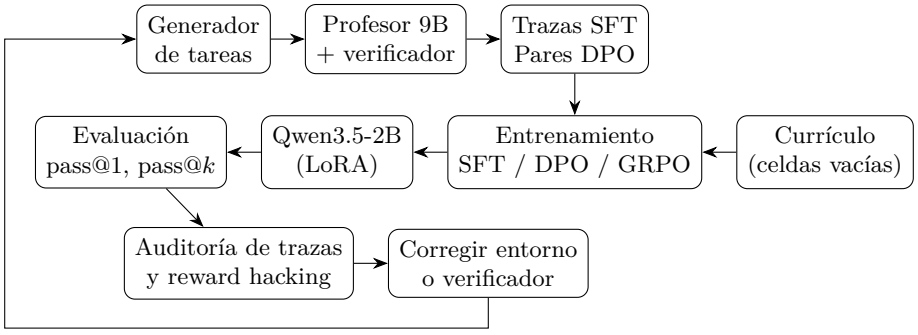


Figura 1: Diagrama de bloques tentativo. El verificador también es la recompensa del RL, y el currículo decide qué tareas ve el entrenamiento.

y en horas de GPU en la A6000, fijado según lo que permita el GRPO, que es el más caro. SFT→RLVR reparte el mismo B entre sus dos etapas.

- *Curvas contra cómputo acumulado*: se evalúan checkpoints a marcas fijas de B y se grafica `pass@1` contra tokens y horas de GPU, de modo que la igualdad de cómputo se lee en la curva y no depende de un único punto de parada.
- *Configuración común*: mismo rango LoRA, mismo largo máximo de respuesta y mismo presupuesto de evaluación en todos los caminos.
- *Costo de datos por separado*: las trazas SFT y los pares DPO requieren inferencia del profesor (9B), que RLVR no necesita. Ese costo se reporta aparte y no se descuenta de B.

[Por decidir: valor de B, y si el costo del profesor se incluye en alguna comparación]

Sin señal en RL. Si el modelo nunca acierta, no hay gradiente. *Mitigación:* concentrarse en la zona aprendible y, si hace falta, calentar con SFT.
Verificador explotable. *Mitigación:* casos de prueba adversariales, inspección de trazas y reentrenamiento con el verificador corregido.
El currículo no aporta o se estanca. Un umbral mal calibrado puede detener el avance o saltar demasiado rápido. *Mitigación:* calendario fijo como alternativa y registro de la dificultad muestreada durante el entrenamiento.
Harness de otros grupos no disponibles o incompatibles. *Mitigación:* elegir al menos dos candidatos, verificar la interfaz y que el modelo base tenga señal antes de comprometerse, y dejar esta evaluación como secundaria si no se puede hacer a tiempo.
Cómputo insuficiente para varias semillas. *Mitigación:* priorizar semillas en los caminos 3 y 4 y en las variantes de currículo, y reducir ablations.

6 Cronograma

Fecha	Hito
11 nov	Entrega 1: esta propuesta.
11–25 nov	Línea base final (cluster), datos SFT/DPO, entrenamiento de SFT y DPO, primeras curvas de recompensa GRPO.
25 nov	Entrega 2: informe de avance (línea base, implementación inicial, curvas de recompensa). Elegir los harness externos.
25 nov – 9 dic	Caminos 3 y 4 completos, currículo y ablations, auditoría manual, redacción del paper y del póster.
9 dic	Entrega 3: paper (7 páginas), póster y video.

Referencias

[1] Reem Aleithan, Haoran Xue, Mohammad Mahdi Mohajer, Elijah Nnorom, Gias Uddin, and Song Wang. SWE-Bench+: Enhanced coding benchmark for LLMs. *arXiv preprint arXiv:2410.06992*, 2024.

[2] Yoshua Bengio, Jérôme Louradour, Ronan Collobert, and Jason Weston. Curriculum learning. In *International Conference on Machine Learning (ICML)*, 2009.

[3] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, et al. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*, 2021.

[4] DeepSeek-AI. DeepSeek-R1: Incentivizing reasoning capability in LLMs via reinforcement learning. *arXiv preprint arXiv:2501.12948*, 2025.

[5] Edward J. Hu, Yelong Shen, Phillip Wallis, et al. LoRA: Low-rank adaptation of large language models. In *International Conference on Learning Representations (ICLR)*, 2022.

[6] Kimi Team. Kimi k1.5: Scaling reinforcement learning with LLMs. *arXiv preprint arXiv:2501.12599*, 2025.

[7] Nathan Lambert, Jacob Morrison, Valentina Pyatkin, et al. TÜLU 3: Pushing frontiers in open language model post-training. *arXiv preprint arXiv:2411.15124*, 2024.

[8] Zichen Liu, Changyu Chen, Wenjun Li, et al. Understanding R1-Zero-Like training: A critical perspective. *arXiv preprint arXiv:2503.20783*, 2025.

[9] Pontificia Universidad Católica de Chile. Proyecto de Investigación: IIC-3694 Inteligencia Artificial Generativa, 2026. Enunciado del curso.

[10] Rafael Rafailov, Archit Sharma, Eric Mitchell, Stefano Ermon, Christopher D. Manning, and Chelsea Finn. Direct preference optimization: Your language model is secretly a reward model. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.

[11] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017.

[12] Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, et al. DeepSeekMath: Pushing the limits of mathematical reasoning in open language models. *arXiv preprint arXiv:2402.03300*, 2024.

[13] Joar Skalse, Nikolaus H. R. Howe, Dmitrii Krasheninnikov, and David Krueger. Defining and characterizing reward gaming. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.

[14] Zafir Stojanovski, Oliver Stanley, Joe Sharratt, Richard Jones, Abdulhakeem Adefioye, Jean Kaddour, and Andreas Köpf. Reasoning gym: Reasoning environments for reinforcement learning with verifiable rewards. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2025.

[15] Leandro von Werra, Younes Belkada, Lewis Tunstall, et al. TRL: Transformer reinforcement learning. `https://github.com/huggingface/trl`, 2020.

[16] Zhiheng Xi, Yiwen Ding, Wenxiang Chen, Boyang Hong, et al. AgentGym: Evaluating and training large language model-based agents across diverse environments. In *Proceedings of the Association for Computational Linguistics*, 2025.

[17] Qiying Yu, Zheng Zhang, Ruofei Zhu, et al. DAPO: An open-source LLM reinforcement learning system at scale. *arXiv preprint arXiv:2503.14476*, 2025.

[18] Yang Yue, Zhiqi Chen, Rui Lu, et al. Does reinforcement learning really incentivize reasoning capacity in LLMs beyond the base model? *arXiv preprint arXiv:2504.13837*, 2025.